LAB 2 / SRE INCIDENT RESPONSE

Canary Deployment Outage INC-94821

Student experiment booklet | CloudRetail Production Incident Simulation | 90-120 minutes

Investigator Name: _____ **Lab ID:** _____ **Section:** _____

Incident Briefing: Incident INC-94821

It is 03:42 UTC. You are the on-call Site Reliability Engineer (SRE) for CloudRetail Inc., an e-commerce platform processing thousands of live customer orders per minute. Ten minutes ago, the automated CI/CD pipeline deployed a canary release of Platform Release v2.4.1 into the production Kubernetes cluster. Synthetic health checks failed immediately, customer checkouts crashed, and automated PagerDuty alert #INC-94821 paged your team:

PAGERDUTY / INCIDENT ALERT PAYLOAD

```
[ALERT] 03:41:20 PagerDuty INC-94821 (P1 - Outage)
Service Health Probe Failed: HTTP 503 Service Unavailable
Canary Rollout Halted. Production Ingress Diverted.
```

The morning shift engineering team arrives in two hours. Your job is not to write complex new code. Your job is to act as an incident investigator:

1. Audit History: Re-examine shell history to audit past commands and recall investigative pipelines.
2. Inspect Logs: Navigate large build/deploy logs with less to identify the failing microservice.
3. Slice Tickets: Extract delimited issue tracker records with cut to isolate critical bugs.
4. Order Priorities: Sort tickets with sort by severity, avoiding destructive file truncation traps.
5. Aggregate Workloads: Group assignees with uniq -c and observe pipeline order sensitivity.
6. Detect Drift: Compare release configurations with diff to isolate the timeout regression.
7. Repair & Handover: Fix the configuration with nano and complete the verified handover report.

Environment Setup & Forensic Workspace

This lab is pre-configured to run inside a GitHub Codespace or standard Linux container. From the repo root, initialize a fresh attempt:

BASH / TERMINAL COMMAND

```
python3 lab/start.py
```

Enter your attempt directory (e.g., cd lab-work/attempt-1), then run pwd and inspect your evidence files:

My lab root (full path printed by pwd): _____

Folder / File	Forensic Purpose & Incident Context
build.log	Raw CI/CD container build and deployment logs (180+ lines).
tickets.txt	Pipe-delimited incident tickets from the production issue tracker.
config-v1.env	Known good configuration from stable release v2.4.0.
config-v2.env	Broken configuration deployed in canary release v2.4.1.
config-fixed.env	Working copy where you will apply configuration fixes.
handover.md	The template handover document to complete for the morning shift.
sandbox/	A safe scratchpad directory for high-risk experiments.

How to work: Predict, try, inspect, and explain. Begin every experiment at your lab root. Type one command at a time. Do not write full transcripts; record specific forensic observations on the prompt lines.

EXPERIMENT 1 / 10 MINUTES

Find a command you used before

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here.

The Unix shell maintains an in-memory ring buffer of executed commands, synchronized to ~/.bash_history. During high-pressure outages, retyping long commands introduces typos and wastes precious time. Engineers use history to audit past actions and recall complex command pipelines instantly.

1A | Make a forensic trail

Run these three simulated investigative commands in your attempt directory to establish a known history trail:

BASH / TERMINAL COMMAND

```
curl -s http://localhost:8080/healthz
grep -i "timeout" build.log
tail -n 15 build.log
history 10
```

history 10 displays the last 10 commands with history numbers.

Which history number appears beside your grep search? _____

1B | Filter the history list

Search your history list by piping it into grep:

BASH / TERMINAL COMMAND

```
history | grep -F "build.log"
```

This filters your history buffer for commands referencing build.log.

Write one earlier matching command, excluding the history search itself:

_____

1C | Search interactively with Ctrl+R, then edit

At an empty Bash prompt, press Ctrl+R. The prompt switches to (reverse-i-search). Type timeout slowly. The matching grep command appears.

1. Press Right Arrow (or Ctrl+E) to accept the command onto your command line without executing it.
2. Use the arrow keys and Backspace to add -n to display line numbers: grep -i -n "timeout" build.log.
3. Press Enter to execute the edited command.

What did your recalled, edited command print?

_____

1D | Safe search abort with Ctrl+G

Press Ctrl+R, type healthz, then press Ctrl+G (or Ctrl+C). This cancels reverse-i-search and restores an empty prompt without running any command.

Student Task 1 | Record investigative pipeline

Record your forensic search command into the incident audit log:

BASH / TERMINAL COMMAND

```
echo "grep -E '(FATAL|ERROR|CRITICAL)' build.log" >> investigation.log
```

Verify the audit log: cat investigation.log.

EXPERIMENT 2 / 12 MINUTES

Inspect logs without hanging your terminal

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here.

When inspecting production logs spanning thousands of lines, never run cat. Running cat blows past your terminal scrollback buffer, floods your screen, and risks locking your shell. The less pager lets you navigate files of any size interactively with zero memory overhead.

Key in less	Action	Purpose in Incident Triage
Space / b	Page Forward / Backward	Scroll down / up one full window
Down / Up	Line Down / Up	Step through log lines individually
g / G	Jump to Start / Jump to EOF	Jump to initial boot (Line 1) / jump to latest crash events
/pattern	Search Forward	Search forward for error signatures (e.g., /FATAL)
n / N	Next / Previous Match	Advance to next / previous instance of search pattern
q	Quit	Exit pager cleanly and return to shell prompt

2A | Inspect the pipeline crash at EOF (G)

Open the deployment log in less:

BASH / TERMINAL COMMAND

```
less build.log
```

Press G (Shift+G) to jump directly to the bottom of the log to see where the container terminated.

What exit status code is reported at EOF? _____ Press g to return to Line 1.

2B | Search for fatal error signatures (/FATAL)

Search forward for fatal errors by typing:

BASH / TERMINAL COMMAND

```
/FATAL
```

Press Enter. The cursor jumps directly to the fatal database crash report.

What error message appears beside the first fatal entry?

_____

2C | Navigate stack trace matches (n and N)

Press n to advance to the next fatal log entry. Notice the stack trace. Press N to return to the previous match. Press q to exit less.

Student Task 2 | Identify the failing microservice

From your less inspection, determine which microservice crashed with FATAL: Database connection pool failed to initialize: frontend-web, auth-service, inventory-api, or checkout-service?

Identified failing microservice: _____

Save the failing service name into failing_service.txt:

BASH / TERMINAL COMMAND

```
echo "checkout-service" > failing_service.txt
```

EXPERIMENT 3 / 10 MINUTES

Choose the fields you need with cut

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here.

In production incidents, engineers frequently query tabular data exports (CSV, TSV, or pipe-delimited database dumps). The cut utility isolates specific vertical columns (fields) from text streams.

3A | Inspect delimited tickets with head

Inspect the first few records in tickets.txt:

BASH / TERMINAL COMMAND

```
head -n 5 tickets.txt
```

BASH / TERMINAL COMMAND

```
TICKET_ID|SERVICE|SEVERITY|ASSIGNEE|STATUS|TITLE
CR-102|checkout-service|1|alice|OPEN|Checkout gateway connection timeout in canary
CR-45|auth-service|2|devon|RESOLVED|Update RS256 token signing rotation keys
CR-18|checkout-service|2|bob|OPEN|Database pool max connections sizing review
CR-304|inventory-api|3|elena|IN_PROGRESS|Slow warehouse query on holiday catalog cache
```

Each record has 6 fields delimited by |: $1=TICKET_ID, $2=SERVICE, $3=SEVERITY, $4=ASSIGNEE, $5=STATUS, $6=TITLE.

3B | Field isolation with cut -d "|"

Extract only Ticket ID and Assignee (Fields 1 and 4):

BASH / TERMINAL COMMAND

```
cut -d "|" -f 1,4 tickets.txt
```

[i] KEY PRINCIPLE: Crucial Best Practice: Always quote your delimiter (-d "|" or -d ','). In Unix shells, the unquoted pipe character | is interpreted as a stream pipe! Running cut -d | -f 1 causes a shell syntax error.

From the first data record (CR-102): Field 1 = _____ Field 4 = _____

3C | Select field ranges (-f 2-4)

Extract Service, Severity, and Assignee:

BASH / TERMINAL COMMAND

```
cut -d "|" -f 2-4 tickets.txt
```

Notice that cut preserves the delimiter between selected fields.

Student Task 3 | Isolate checkout-service tickets

Extract all ticket records belonging strictly to checkout-service into checkout_tickets.txt:

BASH / TERMINAL COMMAND

```
grep "checkout-service" tickets.txt > checkout_tickets.txt
```

Verify the record count: wc -l checkout_tickets.txt.

Number of checkout-service ticket records isolated: _____ (Expected: exactly 7)

EXPERIMENT 4 / 15 MINUTES

Put records in order & avoid file destruction

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here.

Raw data streams rarely arrive ordered by urgency. The sort command arranges lines in lexicographical or numerical order.

4A | The Text vs. Numeric Sort Trap

Consider the numbers: 2, 9, 18, 102.

- When sorted as text (ASCII order): 102, 18, 2, 9 (because "1" precedes "2" in ASCII!).
- When sorted numerically with -n: 2, 9, 18, 102.

Always use -n when sorting severity levels, port numbers, or error counts!

Text order first item: _____ **Numeric order first item:** _____

4B | REQUIRED TRAP: The File Redirection Clobber Trap (sort file > file)

A devastating mistake made by junior developers is attempting to sort a file in place using output redirection:

o o o BASH / TERMINAL COMMAND

```
# WARNING: DO NOT RUN THIS ON PRODUCTION FILES!
sort tickets.txt > tickets.txt
```

Why does this DESTROY your file?

When you type > filename, the shell processes output redirection before the sort command even starts!

1. The shell opens tickets.txt with flags O_WRONLY | O_CREAT | O_TRUNC.
2. The O_TRUNC flag immediately truncates the file to 0 bytes!
3. The shell then forks and executes sort tickets.txt.
4. sort opens tickets.txt, finds 0 bytes, reads 0 lines, and writes 0 lines back!
5. Result: Your entire data file is permanently wiped out!

Safe Sandbox Demonstration:

o o o BASH / TERMINAL COMMAND

```
wc -l sandbox/tickets_copy.txt
sort sandbox/tickets_copy.txt > sandbox/tickets_copy.txt
wc -l sandbox/tickets_copy.txt
```

File size in sandbox after same-file redirection: _____ **bytes (0 lines!)**

4C | The Two Safe In-Place Solutions

1. Redirect to temporary file, then move: sort input.txt > input.tmp && mv input.tmp input.txt
2. Use sort's -o flag: sort -o input.txt input.txt (reads entire file into memory before writing!)

Student Task 4 | Sort checkout tickets by Severity

Sort checkout_tickets.txt by Severity (Column 3) in ascending numerical order (Severity 1 critical bugs first):

o o o BASH / TERMINAL COMMAND

```
sort -t "|" -k 3,3n checkout_tickets.txt > sorted_tickets.txt
```

Inspect sorted_tickets.txt.

Highest severity ticket at Line 1: _____ **(Expected: CR-102 / Severity 1)**

EXPERIMENT 5 / 15 MINUTES

When does a duplicate disappear?

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here.

The uniq utility reports or omits repeated lines. However, it comes with a fundamental rule that catches many engineers off-guard:

[i] KEY PRINCIPLE: uniq ONLY detects and merges ADJACENT duplicate lines.

If duplicate lines are separated by other lines, uniq treats each occurrence as an independent item!

5A | Inspect raw assignee stream

Extract the raw assignees list from checkout_tickets.txt:

BASH / TERMINAL COMMAND

```
cut -d "|" -f 4 checkout_tickets.txt
```

The raw stream prints: alice, bob, alice, charlie, bob, alice, bob. Notice that alice appears 3 times, but separated by bob and charlie!

5B | REQUIRED EXPERIMENT: Running uniq -c WITHOUT prior sorting

Run:

BASH / TERMINAL COMMAND

```
cut -d "|" -f 4 checkout_tickets.txt | uniq -c
```

Observed output: Every single line is counted as 1! Alice appears 3 separate times with count 1. Duplicates were never merged because they were not adjacent!

Did unsorted uniq calculate true workloads? _____ **Why?** _____

5C | The correct pipeline: sort | uniq -c

Sort the stream first so identical lines become contiguous:

BASH / TERMINAL COMMAND

```
cut -d "|" -f 4 checkout_tickets.txt | sort | uniq -c
```

Observed output:

BASH / TERMINAL COMMAND

```
   3 alice
   3 bob
   1 charlie
```

Because sort grouped all alice records together, uniq -c successfully calculated the true workload!

Alice tickets: _____ **Bob tickets:** _____ **Charlie tickets:** _____ **Total:** _____

5D | Comparing uniq | sort vs sort | uniq

What happens if you pipe uniq into sort (uniq | sort)?

The unmerged duplicates survive uniq, and sort groups the unmerged counts: three separate rows of 1 alice!

This proves that Unix pipelines are non-commutative: changing tool order alters the semantic result.

Student Task 5 | Generate authoritative owners summary

Generate the engineer workload summary for checkout-service:

BASH / TERMINAL COMMAND

```
cut -d "|" -f 4 checkout_tickets.txt | sort | uniq -c | sed 's/^[ \t]*//' > owners_summary.txt
```

Verify owners_summary.txt: cat owners_summary.txt.

EXPERIMENT 6 / 12 MINUTES

What changed between release versions?

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here.

In production incidents, over 75% of outages are caused by configuration changes rather than binary code defects. The diff tool compares two text files line-by-line and highlights changes, additions, and deletions.

6A | Understanding Diff Formats

- Standard Diff (diff f1 f2): < from f1, > from f2, c change, a addition, d deletion. 14c14 means line 14 changed.
- Unified Diff (diff -u f1 f2): Standard format used by Git and software patches. --- original, +++ new, @@ hunk header, - line removed, + line added.

6B | Compare stable release against broken canary

Compare stable config-v1.env against broken canary config-v2.env:

BASH / TERMINAL COMMAND

```
diff -u config-v1.env config-v2.env
```

Unified diff output:

CONFIG / UNIFIED DIFF OUTPUT

```
@@ -14,3 +14,3 @@
 DB_POOL_MAX=50
-DB_TIMEOUT_MS=5000
+DB_TIMEOUT_MS=50

@@ -19,3 +19,3 @@
 PAYMENT_GATEWAY_URL=https://api.stripe.com/v1
-PAYMENT_RETRY_ATTEMPTS=3
+PAYMENT_RETRY_ATTEMPTS=0
```

6C | Root Cause Forensic Isolation

Look at DB_TIMEOUT_MS:

- In config-v1.env, the connection timeout was 5000 (5,000 milliseconds = 5 seconds).
- In config-v2.env, a developer accidentally dropped two zeros, configuring 50 (50 milliseconds)!
- Under real cloud network conditions, a database TCP handshake with TLS negotiation takes 80-120ms. A 50ms timeout guaranteed that 100% of connection attempts failed instantly!
- Look at PAYMENT_RETRY_ATTEMPTS: Dropped from 3 to 0, preventing transient network retries.

Stable DB_TIMEOUT_MS: _____ **Canary DB_TIMEOUT_MS:** _____

Root cause explanation: _____

Student Task 6 | Capture configuration drift to config.diff

Save the configuration drift analysis:

BASH / TERMINAL COMMAND

```
diff -u config-v1.env config-v2.env > config.diff
```

Verify: head -n 15 config.diff.

EXPERIMENT 7 / 15 MINUTES

Repair configuration & assemble handover

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here.

When connecting to headless cloud servers, containers, or bastion hosts via SSH, graphical editors are unavailable. nano is the universal, modeless in-terminal editor installed across Linux environments.

Shortcut	Action	Purpose in Incident Response
Ctrl+O, Enter	WriteOut (Save)	Saves changes to disk. Prompts for filename; press Enter to confirm.
Ctrl+X	Exit	Closes nano. If unsaved changes exist, prompts Y/N to save.
Ctrl+W	Where Is (Search)	Searches forward for a string inside the file.
Ctrl+K / Ctrl+U	Cut Text / Uncut (Paste)	Cuts current line into buffer / Pastes cut buffer at cursor position.
Ctrl+C	Cursor Position	Displays current line and column numbers.
Alt+U	Undo	Reverses the last editing action.

7A | Repair the working configuration in nano

Open your working configuration copy (config-fixed.env) in nano:

BASH / TERMINAL COMMAND

```
nano config-fixed.env
```

1. Press Ctrl+W, search for DB_TIMEOUT_MS, change 50 to 5000.
2. Press Ctrl+W, search for PAYMENT_RETRY_ATTEMPTS, change 0 to 3.
3. Press Ctrl+O, then Enter to save. Press Ctrl+X to exit.
4. Verify your fix with diff: diff -u config-v1.env config-fixed.env.

What did the verification diff print? _____ (Empty = identical to stable v1!)

7B | Assemble the Release Handover Report (handover.md)

Open handover.md in nano:

BASH / TERMINAL COMMAND

```
nano handover.md
```

1. Replace [YOUR NAME HERE] with your name or title (e.g., SRE Lead Investigator).
2. Replace [REPLACE WITH FAILING SERVICE IDENTIFIED IN BUILD.LOG] with checkout-service.
3. In Section 2, paste the error logs from build.log. Set Bad Value to 50 and Stable Value to 5000.
4. In Section 3, paste the contents of sorted_tickets.txt.
5. In Section 4, paste the contents of owners_summary.txt: 3 alice, 3 bob, 1 charlie.
6. Save with Ctrl+O, Enter, and exit with Ctrl+X.

Student Task 7 | Confirm handover.md completion

Verify all placeholders removed: grep -F "[REPLACE" handover.md (must return 0 matching lines).

EXIT CHECK & COMMAND REFERENCE / 10 MINUTES

Automated verification & command cheatsheet

Start: From your active attempt directory, run: python3 ../../lab/check.py

8A | Automated verification check (python3 ../../lab/check.py)

Run the automated verification script:

BASH / TERMINAL COMMAND

```
python3 ../../lab/check.py
```

check.py is strictly read-only and validates all 7 tasks:

- [PASS] Task 1 (less): Failing service correctly identified as checkout-service
- [PASS] Task 2 (cut/grep): Isolated tickets for checkout-service (7 records)
- [PASS] Task 3 (sort): Tickets sorted by Severity in priority order
- [PASS] Task 4 (uniq): Owner summary properly sorted and aggregated with counts
- [PASS] Task 5 (diff): config.diff captures configuration drift (DB_TIMEOUT_MS)
- [PASS] Task 6 (nano): config-fixed.env restored with DB_TIMEOUT_MS=5000 and PAYMENT_RETRY_ATTEMPTS=3
- [PASS] Task 7 (handover.md): Report fully populated and placeholders removed

My verification score: _____ / 7 checks passed | Verification status: [PASS]

8B | Short Understanding & Exit Check

1. Why does sort file > file destroy data?

_____

2. Why must sort precede uniq -c in workload aggregation?

_____

3. What does an empty diff output prove between two files?

_____

8C | Quick Command Reference (Keep beside your terminal)

Tool	Syntax / Keybinding	Purpose & SRE Workflow Context
history	history 10 / Ctrl+R	List recent commands / Reverse incremental search
less	Space, b, g, G, /pattern, q	Interactive log navigation without terminal pollution
cut	cut -d "|" -f 1,4	Extract vertical fields (always quote delimiter)
sort	sort -t "|" -k 3,3n / sort -o f f	Numerical field sort / Safe in-place sort
uniq	sort | uniq -c	Aggregate adjacent lines (sorting first is mandatory!)
diff	diff -u file1 file2	Unified configuration comparison (- removed, + added)
nano	Ctrl+O, Enter / Ctrl+X	In-terminal editing, saving, and exiting

LAB MISSION COMPLETE: Incident INC-94821 Successfully Mitigated & Handed Over